

Overview

Mobile Build Blueprint - webapp-main

Scope and resolved revision

Resolved main commit: 368cec4aff9f293a2564eb6c35f3d9045b343c2c

Requested cutoff: Jul 10, 2026 at 18:00 IST

Repository: kashyapempiricinfotech-sys/webapp-main

Method: source-only inspection. The app was not run and no live application endpoint was called.

The selected commit is the last commit at or before the cutoff; GitHub shows the next two main-branch commits after the cutoff, and `compare_commits` confirms the selected SHA is their main-branch ancestor.

Recommendation

Use React Native with Expo (inferred recommendation) to preserve the existing React/TypeScript mental model while replacing the Next.js UI and browser-specific services with mobile navigation, secure token storage, native deep links, push permissions, and a typed API client.

Tab guide

- Architecture - runtime stack, business modules, registered endpoints, auth, integrations, and client/server contract gaps.
- Screen Inventory - all six code-backed screens and four reusable UI patterns, with purpose and reuse counts.
- Mobile Stack - framework, native bridge, testing, and CI/CD alternatives with current vendor plan pricing.
- User Flow - evidence-backed and inferred screen-to-screen movement, including checkout return paths and role/feature gates.
- Data Model - Prisma entities, TypeScript contracts, relationships, and schema/contract mismatches.
- Build Plan - four buildable phases across Sprint 1 through Sprint 6, dependencies, and completion gates.
- Risks & Open Questions - decisions and gaps that must be resolved before release.
- Secrets found (review) - file-and-line pointers only; no credential values are reproduced.
- Traceability & Sources - repository scope, evidence conventions, and current vendor source links.

Architecture

Architecture

Runtime stack

- Next.js 15.4 and React 19.1 render the web screens. Evidence: package.json:16-18; app/*/page.tsx.
- Express 5.1 exposes a separate server on port 3001. Evidence: package.json:14; server/index.ts:11-26.
- TypeScript 5.8 and tsx 4.20 support type-checking and server execution. Evidence: package.json:7-9,23-24.
- Vitest 3.2 is declared, but no test file is present in the commit tree (inferred gap). Evidence: package.json:9,25; complete commit file list.
- PostgreSQL is the Prisma datasource. Evidence: packages/data/prisma/schema.prisma:5-8.

Business logic modules

Active or registered

- Authentication: client sign-in and bearer header helpers call the JWT endpoints. Evidence: src/services/auth.ts:3-10; server/routes/auth.ts:6-12; server/middleware/requireAuth.ts:4-12.
- Invoices and checkout: client list/checkout calls map to authenticated Express routes; checkout delegates to Stripe. Evidence: src/services/invoices.ts:4-12; server/routes/invoices.ts:6-9; src/integrations/stripe.ts:5-11.
- Notification preferences: authenticated GET and PUT routes are registered. Evidence: server/index.ts:18; server/routes/notifications.ts:3-5.
- Analytics startup: PostHog startup is called during server bootstrap. Evidence: server/index.ts:2,13; src/integrations/posthog.ts:3-8.
- Conditional error reporting: Sentry startup is called, but initialization returns early unless enabled. Evidence: server/index.ts:3,14; src/integrations/sentry.ts:3-6.

Present but not wired

- reportingAggregator is explicitly described as unwired dead code. Evidence: src/services/reportingAggregator.ts:3-5.
- cloudBootstrap is declared but has no import or call in the inspected tree. Evidence: src/config/cloud.ts:1-4; complete commit file list.

API endpoint inventory

Public

- GET /api/health - framework handler; returns an OK payload. Evidence: api/health.ts:1-3.
- POST /api/auth/login - creates a JWT and returns token plus user. Evidence: server/index.ts:16; server/routes/auth.ts:6-10.
- POST /api/auth/refresh - returns a fixed fixture token rather than implementing refresh validation. Evidence: server/routes/auth.ts:12.
- GET /api/catalog - manifest route with no auth; returns a starter catalog item. Evidence: server/routes/manifest.ts:5-7; server/index.ts:21-23.

Bearer JWT required

- GET /api/invoices - returns an empty array in the inspected implementation. Evidence: server/index.ts:17; server/routes/invoices.ts:6.
- POST /api/invoices/:invoiceId/checkout - creates a Stripe Checkout Session for a fixed minor-unit amount and returns its URL. Evidence: server/routes/invoices.ts:7-9; src/integrations/stripe.ts:5-11.
- GET /api/notifications/preferences - returns paymentReminders. Evidence: server/index.ts:18; server/routes/notifications.ts:4.
- PUT /api/notifications/preferences - coerces and returns paymentReminders. Evidence: server/routes/notifications.ts:5.
- GET /api/audit-events - manifest route requiring bearer auth; returns an empty array. Evidence: server/routes/manifest.ts:7; server/index.ts:21-23.

Legacy cookie session

- GET /api/legacy/profile - returns the legacy session user id or null. Evidence: server/index.ts:19; server/routes/legacy.ts:5-6.
- POST /api/legacy/logout - clears the legacy session and returns 204. Evidence: server/routes/legacy.ts:5-7.

Auth flows

Current JWT flow

- Login posts email and password to /api/auth/login. Evidence: src/services/auth.ts:3-6.
- Server derives role from the submitted email suffix, signs a token, and returns user data. Evidence: server/routes/auth.ts:6-10.
- Protected routes strip Bearer from Authorization and verify the token. Evidence: server/middleware/requireAuth.ts:4-12.

- Mobile requirement (inferred): store access and refresh material in Keychain/Keystore-backed storage, centralize 401 retry, and define logout/revocation behavior.

Legacy flow

- Two live endpoints still use cookie-session middleware. Evidence: `server/auth/legacySession.ts:3-7`; `server/routes/legacy.ts:5-7`.
- Mobile requirement (inferred): confirm whether mobile must support the legacy cookie flow; otherwise deprecate it behind a documented migration window.

Third-party integrations

- Stripe is active for invoice checkout; success and cancel return through the webapp:// scheme. Evidence: `src/integrations/stripe.ts:3-10`.
- PostHog initialization is active at server startup; `invoice_opened` tracking is defined but not called in the inspected tree. Evidence: `server/index.ts:13`; `src/integrations/posthog.ts:3-8`.
- Sentry is wired but disabled unless `SENTRY_ENABLED` is true. Evidence: `src/integrations/sentry.ts:3-6`; `.env.example:4`.

Client/server contract gaps

- `registerPushToken` calls `POST /api/notifications/installations`, but no matching registered server route exists. Evidence: `src/services/notifications.ts:5-6`; `server/routes/notifications.ts:3-5`; `server/index.ts:16-23`.
- `saveSettings` calls `PUT /api/settings`, but no `/api/settings` router is registered; the server exposes `PUT /api/notifications/preferences` instead. Evidence: `src/services/settings.ts:3-4`; `server/index.ts:16-23`; `server/routes/notifications.ts:5`.
- The login request does not set `Content-Type: application/json` (inferred interoperability risk). Evidence: `src/services/auth.ts:3-5`; `server/index.ts:12`.
- Prisma models exist, but the registered route handlers do not import or query a Prisma client (inferred persistence gap). Evidence: `packages/data/prisma/schema.prisma:1-32`; `server/routes/*.ts`.

Screen Inventory

Screen Inventory

Screens

Sign in

Purpose: submits demo credentials to the JWT login service.

Route: /login.

Reuse: PrimaryCta once.

Evidence: app/login/page.tsx:1-6; src/services/auth.ts:3-6.

Mobile note (inferred): replace embedded demo credentials with controlled inputs, validation, loading/error states, and secure session persistence.

Dashboard

Purpose: displays an account snapshot and a Review invoices call to action.

Route: /dashboard.

Reuse: AppShell once; PrimaryCta once.

Evidence: app/dashboard/page.tsx:1-4.

Mobile note (inferred): make the CTA navigate to Invoices and define the authenticated landing behavior.

Invoices

Purpose: displays an invoice card and a telemetry-wrapped Create invoice action.

Route: /invoices.

Reuse: AppShell once; InvoiceCard once; PrimaryCta once through withTelemetry; withTelemetry once.

Evidence: app/invoices/page.tsx:1-9.

Mobile note (inferred): replace demo data with listInvoices, add loading/empty/error states, invoice detail, and checkout handoff.

Notifications center

Purpose: shows payment reminder preferences only when FEATURE_PUSH is true.

Route: /notifications-center.

Reuse: AppShell once.

Evidence: app/notifications-center/page.tsx:1-5; .env.example:5.

Mobile note (inferred): coordinate OS permission prompts, installation-token registration, and backend preference state.

Settings

Purpose: saves payment reminder settings.

Route: /settings.

Reuse: AppShell once; PrimaryCta once.

Evidence: app/settings/page.tsx:1-6; src/services/settings.ts:3-4.

Mobile note (inferred): point the client to the registered preference endpoint and expose actual setting state.

Admin reports

Purpose: displays a restricted revenue report for an admin user and Forbidden otherwise.

Route: /admin-reports.

Reuse: AppShell once on the allowed branch.

Evidence: app/admin-reports/page.tsx:1-6; src/types.ts:1-6.

Mobile note (inferred): enforce admin authorization at API and navigation layers, not only in the screen render.

Reusable component inventory

AppShell

Purpose: main/header/title wrapper.

Reuse count: 5 screens - Dashboard, Invoices, Notifications center, Settings, Admin reports.

Evidence: src/components/AppShell.tsx:1-4; imports in the five screen files.

PrimaryCta (ActionButton alias)

Purpose: generic button with optional onClick.

Reuse count: 4 screens - Login, Dashboard, Invoices indirectly through withTelemetry, Settings.

Evidence: src/components/ActionButton.tsx:1-4; src/components/index.ts:1; the four screen imports.

InvoiceCard

Purpose: displays invoice number and status.

Reuse count: 1 screen - Invoices.

Evidence: src/components/InvoiceCard.tsx:1-4; app/invoices/page.tsx:1,9.

withTelemetry

Purpose: wraps a component in an analytics-marked container.

Reuse count: 1 screen - Invoices.

Evidence: src/components/withTelemetry.tsx:1-6; app/invoices/page.tsx:2,5.

Navigation finding

No navigation component or root layout is present in the inspected commit tree (inferred gap). The screen-to-screen links in the User Flow tab are therefore marked inferred unless a handler or deep link is visible in code.

Mobile Stack

Mobile Stack

Cost assumptions

Prices are rough USD monthly list prices checked on **Aug 3, 2026**. Taxes, usage overages, Apple/Google developer-account fees, and enterprise contracts are excluded.

Cross-platform framework options

Option A - React Native with Expo (recommended, inferred)

Fit: strongest reuse of React, TypeScript, component concepts, service modules, and existing webapp:// deep-link intent. React Native recommends using a framework such as Expo for new apps.

Trade-offs: web JSX must be rebuilt with native components; server-only Next/Node packages cannot move into the client; native-project ownership still matters for payments, notifications, and release signing.

Vendor plan priced: React Native/Expo framework license \$0/month; Expo EAS Free \$0/month for limited builds. Expo EAS Starter is \$19/month if hosted build/update capacity is included.

Sources: <https://reactnative.dev/> and <https://expo.dev/pricing>

Option B - Flutter

Fit: mature single-codebase iOS/Android UI, strong rendering control, and broad platform support.

Trade-offs: requires Dart and a near-total UI/client rewrite; the current React/TypeScript implementation provides less direct reuse.

Vendor plan priced: Flutter open-source SDK, \$0/month.

Source: <https://flutter.dev/>

Native bridge options

Option A - Expo Modules API (recommended, inferred)

Fit: Swift/Kotlin modules with a consistent API, New Architecture support, and low boilerplate; aligns with the recommended Expo app.

Trade-offs: adds the expo package dependency and may require development builds for custom native code.

Vendor plan priced: Expo framework/module tooling \$0/month; EAS Starter \$19/month only if cloud build/update services are used.

Source: <https://docs.expo.dev/modules/overview/>

Option B - React Native Turbo Native Modules

Fit: direct New Architecture path, Codegen type safety, and lower-level access including C++ when required.

Trade-offs: more platform-specific setup and a steeper maintenance burden for this small codebase.

Vendor plan priced: React Native open-source toolchain, \$0/month.

Source: <https://reactnative.dev/architecture/landing-page>

Testing tool options

Option A - Maestro

Fit: readable cross-platform end-to-end flows, local simulators/emulators, CI integration, and a hosted device option.

Trade-offs: local tier requires maintaining test devices/runners; hosted concurrency costs scale per device.

Vendor plan priced: Local Free Open Source, \$0/month; Cloud, \$250 per device per month.

Source: <https://maestro.dev/pricing>

Option B - BrowserStack App Automate Device Cloud

Fit: broad real-device coverage and Appium-compatible automation without owning a device lab.

Trade-offs: materially higher baseline than local Maestro and still needs test authoring/maintenance.

Vendor plan priced: Device Cloud, \$199/month billed annually for one parallel test; Device Cloud Pro, \$249/month billed annually.

Source: <https://www.browserstack.com/pricing>

CI/CD pipeline options

Option A - Expo EAS (recommended, inferred)

Pipeline: pull-request typecheck/unit tests -> EAS preview build -> Maestro smoke flow -> protected production build -> store submit -> staged update/release monitoring.

Fit: shortest path for an Expo-managed React Native app and integrated credentials, builds, submits, updates, and workflows.

Trade-offs: platform coupling and usage overages; the Free tier is too constrained for a multi-sprint team.

Vendor plan priced: Starter, \$19/month plus usage; Production, \$199/month plus usage.

Source: <https://expo.dev/pricing>

Option B - Bitrise

Pipeline: repository trigger -> dependency/typecheck/unit-test workflow -> iOS/Android builds -> Maestro or BrowserStack gate -> signed store deployment.

Fit: mobile-specialized CI with managed macOS/Linux infrastructure and up to three concurrent Starter builds.

Trade-offs: more pipeline assembly than EAS for Expo and a higher small-team baseline.

Vendor plan priced: Starter, \$99/month when billed monthly; Hobby, \$0/month for one user and limited credits.

Source: <https://bitrise.io/pricing>

Recommended baseline (inferred)

Use React Native + Expo, Expo Modules API, unit/component tests plus Maestro Local during development, and EAS Starter through the six-sprint build. Move to EAS Production or add BrowserStack only when build concurrency, real-device coverage, update MAUs, or support requirements justify the spend.

User Flow

User Flow

Primary authenticated journey

1. Launch app -> Sign in. This is the only screen with a coded authentication action. Evidence: app/login/page.tsx:5-6.
2. Sign in -> Dashboard after a successful token response (inferred; no navigation handler is present). Evidence: src/services/auth.ts:3-6; app/dashboard/page.tsx:3-4.
3. Dashboard -> Invoices after Review invoices (inferred; the button has no onClick). Evidence: app/dashboard/page.tsx:3-4.
4. Invoices -> invoice detail/checkout (inferred screen gap). The service and server support listing and checkout, but only a list screen exists. Evidence: src/services/invoices.ts:4-12; server/routes/invoices.ts:6-9; app/invoices/page.tsx:8-9.
5. Checkout -> external Stripe Checkout -> app returns to webapp://invoices/:invoiceId with checkout=success or checkout=cancelled. Evidence: src/integrations/stripe.ts:5-10.
6. Return link -> invoice result/detail state (inferred; no matching mobile or web route is present in the commit tree).

Preferences and notifications

1. Authenticated user -> Settings (inferred navigation).
2. Settings -> save reminder preference. Current client target and registered server target do not match. Evidence: app/settings/page.tsx:5-6; src/services/settings.ts:3-4; server/routes/notifications.ts:5.
3. If FEATURE_PUSH is true, user -> Notifications center (inferred navigation). Evidence: app/notifications-center/page.tsx:3-5; .env.example:5.
4. User grants OS notification permission -> app registers installation token -> preferences control reminders (inferred target flow). Current registration call has no matching server endpoint. Evidence: src/services/notifications.ts:5-6; server/routes/notifications.ts:3-5.

Admin journey

1. Authenticated user with role admin -> Admin reports (inferred navigation). Evidence: src/types.ts:1-6; app/admin-reports/page.tsx:4-6.
2. Non-admin -> Forbidden branch. Evidence: app/admin-reports/page.tsx:5.
3. API authorization for report data is required (inferred); the only audit-events endpoint is role-agnostic after bearer verification. Evidence: server/routes/manifest.ts:7; server/middleware/requireAuth.ts:4-12.

Legacy journey

1. Legacy client -> GET /api/legacy/profile using cookie session.

2. Legacy client -> POST /api/legacy/logout clears the cookie session.

Evidence: server/routes/legacy.ts:4-7; server/auth/legacySession.ts:3-7.

Mobile decision (inferred): exclude this flow unless product confirms an active legacy-client requirement.

Data Model

Data Model

Prisma entities

User

Fields: id (primary key), email (unique), role.

Relations: invoices one-to-many; settings optional one-to-one.

Evidence: packages/data/prisma/schema.prisma:10-16.

Invoice

Fields: id, ownerId, number, totalMinor, currency, status.

Relation: ownerId references User.id.

Evidence: packages/data/prisma/schema.prisma:18-26.

NotificationSetting

Fields: userId (primary key), paymentReminders with default true.

Relation: userId references User.id.

Evidence: packages/data/prisma/schema.prisma:28-32.

Relationship summary

- User 1 -> many Invoice.
- User 1 -> zero or one NotificationSetting.

Evidence: packages/data/prisma/schema.prisma:10-32.

Client contracts

- SessionUser constrains role to user or admin. Evidence: src/types.ts:1-7.
- Invoice constrains currency to USD or INR and status to draft, open, or paid. Evidence: src/types.ts:9-16.

Model gaps

- Prisma stores role, currency, and status as unconstrained String while client TypeScript narrows them (inferred consistency risk). Evidence: packages/data/prisma/schema.prisma:13,23-24; src/types.ts:1,14-15.
- No Prisma client dependency is declared and no route imports a Prisma client (inferred persistence gap). Evidence: package.json:11-25; server/routes/*.ts.
- Invoice checkout ignores stored invoice amount and uses a fixed value (inferred correctness risk). Evidence: server/routes/invoices.ts:7-9.

- Notification preference endpoints return literals/coerced input without persistence.
Evidence: `server/routes/notifications.ts:4-5`.

Build Plan

Build Plan

Phase 1 - Mobile foundation and auth

Sprints: Sprint 1-Sprint 2.

Scope:

- Create React Native + Expo workspace, environment handling, navigation shell, typed API client, and deep-link configuration.
- Implement sign-in UI, bearer session handling, secure Keychain/Keystore storage, refresh/logout rules, and authenticated route guards.
- Decide and isolate legacy cookie-session compatibility.
- Remove credential-like fixture literals from mobile-delivered code and define secret-management gates.

Depends on: none.

Done:

- iOS and Android debug builds install from CI.
- Sign-in, session restore, 401 handling, and logout pass unit and Maestro smoke tests.
- No secret-like literal reaches a client bundle.
- Legacy support decision is documented and enforced.

Evidence basis: `app/login/page.tsx`; `src/services/auth.ts`; `server/routes/auth.ts`; `server/middleware/requireAuth.ts`; `server/routes/legacy.ts`; Secrets found (review) tab.

Phase 2 - Core account, invoice, and checkout experience

Sprints: Sprint 2-Sprint 4.

Scope:

- Build Dashboard, invoice list, invoice detail/result, reusable shell/button/card primitives, loading/empty/error states, and accessibility behavior.
- Connect invoice list and checkout to the authenticated API.
- Configure and validate Stripe external checkout plus success/cancel deep-link return.
- Replace demo invoice and no-op CTAs with real navigation/state.

Depends on: Phase 1 navigation, API client, and session handling.

Done:

- Dashboard-to-invoice-to-checkout-to-return works on iOS and Android test builds.
- All invoice states are represented and tested.
- Checkout amount comes from an authorized server-side invoice source (inferred required fix).
- Deep links reject malformed or unauthorized invoice targets.

Evidence basis: `app/dashboard/page.tsx`; `app/invoices/page.tsx`; `src/services/invoices.ts`; `server/routes/invoices.ts`; `src/integrations/stripe.ts`; `src/components/*`.

Phase 3 - Preferences, notifications, analytics, and admin

Sprints: Sprint 4-Sprint 5.

Scope:

- Reconcile /api/settings versus /api/notifications/preferences.
- Implement installation-token registration endpoint, OS permission flow, preference persistence, feature-flag behavior, and token lifecycle.
- Define the mobile analytics event taxonomy and wire screen/invoice events.
- Add Sentry only under explicit environment configuration.
- Add admin navigation and server-enforced role checks for report data.

Depends on: Phase 1 session/API foundation and Phase 2 navigation primitives.

Done:

- Preference read/write and push token register/rotate/revoke work against matching server contracts.
- Permission denied/granted flows are tested on both platforms.
- Analytics and error reporting contain no secrets or sensitive payloads and respect environment flags.
- Non-admin users cannot request admin report data.

Evidence basis: app/settings/page.tsx; app/notifications-center/page.tsx;

src/services/settings.ts; src/services/notifications.ts; server/routes/notifications.ts;

src/integrations/posthog.ts; src/integrations/sentry.ts; app/admin-reports/page.tsx.

Phase 4 - Quality, release, and operational readiness

Sprints: Sprint 5-Sprint 6.

Scope:

- Add unit/component tests, Maestro end-to-end flows, CI quality gates, real-device checks, store metadata, signing, staged rollout, monitoring, and rollback runbook.
- Validate accessibility, offline/error recovery, supported OS versions, security controls, and migration from demo fixtures.

Depends on: Phases 1-3 feature-complete test builds.

Done:

- Protected CI produces signed release candidates for both stores.
- Critical user flows pass automated and selected real-device tests.
- Release checklist, monitoring dashboards, incident/rollback steps, and ownership are documented.
- All issues remain To Do until delivery work actually begins.

Evidence basis: package.json:5-25; complete commit tree contains no CI workflow or test files (inferred gaps); Mobile Stack tab.

Risks & Open Questions

Risks & Open Questions

Launch-blocking contract questions

- Which production API base URL and version should mobile use? Current services use browser-relative paths. Evidence: `src/services/auth.ts:4`; `src/services/invoices.ts:5,11`; `src/services/settings.ts:4`.
- Should mobile support legacy cookie sessions, and for how long? Evidence: `server/auth/legacySession.ts:3-7`; `server/routes/legacy.ts:4-7`.
- What is the real refresh-token contract, expiry model, revocation model, and logout behavior? Current refresh returns a fixed fixture token. Evidence: `server/routes/auth.ts:12`.
- Which endpoint is canonical for reminder settings: `/api/settings` or `/api/notifications/preferences`? Evidence: `src/services/settings.ts:4`; `server/routes/notifications.ts:5`.
- What is the server contract for installation token registration and deletion? The client call has no matching server route. Evidence: `src/services/notifications.ts:5-6`; `server/index.ts:16-23`.

Security and authorization

- JWT signing, verification, cookie, token, demo credential, cloud, and bootstrap literals require security review. No values are copied here. Evidence: Secrets found (review) tab.
- Admin authorization is enforced only by a screen-level role branch in inspected code (inferred server-side authorization gap). Evidence: `app/admin-reports/page.tsx:4-6`; `server/middleware/requireAuth.ts:4-12`.
- Mobile must use secure platform storage and avoid logging auth, payment, or push tokens (inferred requirement).

Product and UX

- No root layout or navigation graph exists at the selected commit (inferred scope gap). Evidence: complete commit file list; `app/*/page.tsx`.
- Dashboard and invoice actions are no-ops or demo-backed. Evidence: `app/dashboard/page.tsx:4`; `app/invoices/page.tsx:6,9`.
- No invoice detail screen exists although checkout return links target an invoice-specific path (inferred missing screen). Evidence: `src/integrations/stripe.ts:9-10`; complete app route list.
- Notifications center renders nothing when disabled; fallback UX is undefined. Evidence: `app/notifications-center/page.tsx:3-5`.

Data and operations

- Prisma schema is not connected to the registered routes (inferred persistence gap). Evidence: `packages/data/prisma/schema.prisma`; `server/routes/*.ts`.
- Checkout uses a fixed amount rather than authorized persisted invoice data. Evidence: `server/routes/invoices.ts:7-9`.
- PostHog browser SDK is initialized from Express startup code (inferred runtime compatibility concern requiring validation). Evidence: `server/index.ts:2,13`; `src/integrations/posthog.ts:1-5`.
- No test files, CI workflows, or release configuration are present at the selected commit (inferred delivery gap). Evidence: complete commit file list; `package.json:5-25`.

Decisions to close in Sprint 1

- Production API environment and versioning.
- Supported OS/device matrix.
- Navigation model and authenticated landing screen.
- Legacy auth support.
- Push provider and data-retention policy.
- Analytics consent and event taxonomy.
- App ownership, signing, store accounts, and release approvers.

Secrets found (review)

Secrets found (review)

No credential, token, key, or encoded value is reproduced in this tab. Review and rotate/remove as appropriate.

Credential-like literals and fallback material

- app/login/page.tsx:6 - embedded demo credential material.
- app/settings/page.tsx:6 - embedded session-token-like literal.
- server/auth/legacySession.ts:6 - cookie signing fallback material.
- server/middleware/requireAuth.ts:8 - JWT verification fallback material.
- server/routes/auth.ts:8 - JWT signing fallback material.
- server/routes/auth.ts:12 - fixed token-like response material.
- src/config/cloud.ts:2 - cloud access-key identifier literal.
- src/services/notifications.ts:3 - encoded bootstrap/proof-like literal.

Credential surfaces to verify

- .env.example:1-3 - JWT, Stripe, and analytics credential variable placeholders.
- src/integrations/posthog.ts:4 - analytics key consumption point.
- src/integrations/sentry.ts:5 - error-reporting DSN consumption point.
- src/integrations/stripe.ts:3 - payment secret consumption point.

Required handling (inferred)

- Treat every listed literal as compromised until security review classifies it.
- Remove client-shipped credentials and bootstrap proofs; issue short-lived server-authorized tokens where required.
- Store production secrets only in approved server and CI secret stores.
- Add secret scanning and client-bundle inspection to CI.
- Do not paste any discovered value into Jira, documentation, Teams, build logs, or test fixtures.

Traceability & Sources

Traceability & Sources

Repository evidence

Repository: kashyapempiricinfotech-sys/webapp-main

Resolved commit: 368cec4aff9f293a2564eb6c35f3d9045b343c2c

Commit URL:

<https://github.com/kashyapempiricinfotech-sys/webapp-main/commit/368cec4aff9f293a2564eb6c35f3d9045b343c2c>

Evidence format: repository-relative file:line.

Inspection scope: every file in the selected commit, with routes under api and server/routes reviewed before the rest of the blueprint.

No app execution and no live endpoint access.

Inference convention

Every recommendation, proposed navigation step, future-state requirement, or gap that is not directly implemented is labeled inferred or inferred recommendation.

Current vendor sources

- React Native overview: <https://reactnative.dev/>
- Flutter overview: <https://flutter.dev/>
- Expo pricing: <https://expo.dev/pricing>
- Expo Modules API: <https://docs.expo.dev/modules/overview/>
- React Native New Architecture: <https://reactnative.dev/architecture/landing-page>
- Maestro pricing: <https://maestro.dev/pricing>
- BrowserStack pricing: <https://www.browserstack.com/pricing>
- Bitrise pricing: <https://bitrise.io/pricing>

Pricing notes

Vendor list prices are snapshot estimates, not quotes. Re-check before procurement. Expo and Bitrise can incur usage charges; BrowserStack prices shown are annual-billing monthly equivalents; Maestro Cloud scales by maximum concurrent devices.